IETF-126 Hackathon

I2ICF Side Meeting

Interface to In-Network Computing Functions (I2ICF) Project

July 21, 2026, Vienna

Champion: Jaehoon (Paul) Jeong

Members: Mose Gu, Yoseop Ahn, Jisuk Chae, **EunJin Hwang**, Kihyeon Shim, KyeongHoon Jeong, Joonsang Park, YoonJun Choi, Jisol Min

Department of Computer Science and Engineering at SKKU

Email: {pauljeong, ahnjs124, rna0415, sjw6136, sue030124, wangxudong28, eunijhwang}@skku.edu

IETF-126 Interface to In-Network Computing Functions (I2ICF)

IETF-126 Interface to In-Network Computing Functions (I2ICF) Project

Professors:

- Jaehoon Paul Jeong (SKKU)
- Yong-Geun Hong (DJU)
- Joo-Sang Youn (DEU)

Researchers:

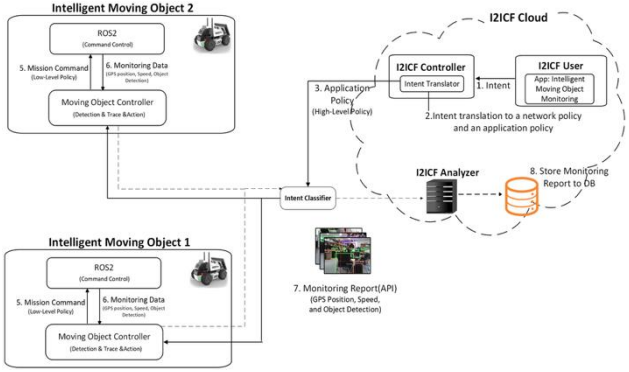
- Kehan Yao (China Mobile)
- Byoungman Robert An (KETI)
- Jung-Soo Park (ETRI)
- Yunchul Choi (ETRI)

Students:

- Mose Gu (SKKU)
- Yoseop Ahn (SKKU)
- Jisuk Chae (SKKU)
- Kihyun Shim (SKKU)
- Eunjin Hwang (SKKU)
- Keonghun Jeong (SKKU)
- Yoonjun Choi (SKKU)
- Joonsang Park (SKKU)
- Jisol Min (SKKU)

Champion: Jaehoon Paul Jeong (SKKU)

I2ICF Framework



Objectives

- To demonstrate an intent-driven, closed-loop safety pipeline in which a moving object receives an intent, this I2ICF framework (i) performs on-board camera-based object detection and camera–LiDAR calibration/fusion for distance estimation, (ii) streams detections to a cloud server, and (iii) executes real-time stop/avoidance for obstacles and pedestrians.

Future Work

- To optimize End-to-End performance (e.g., inference accuracy, and inference latency) and extend to two robot cars that exchange on-board inference data to cooperatively plan collision-free, optimal paths when approaching each other.

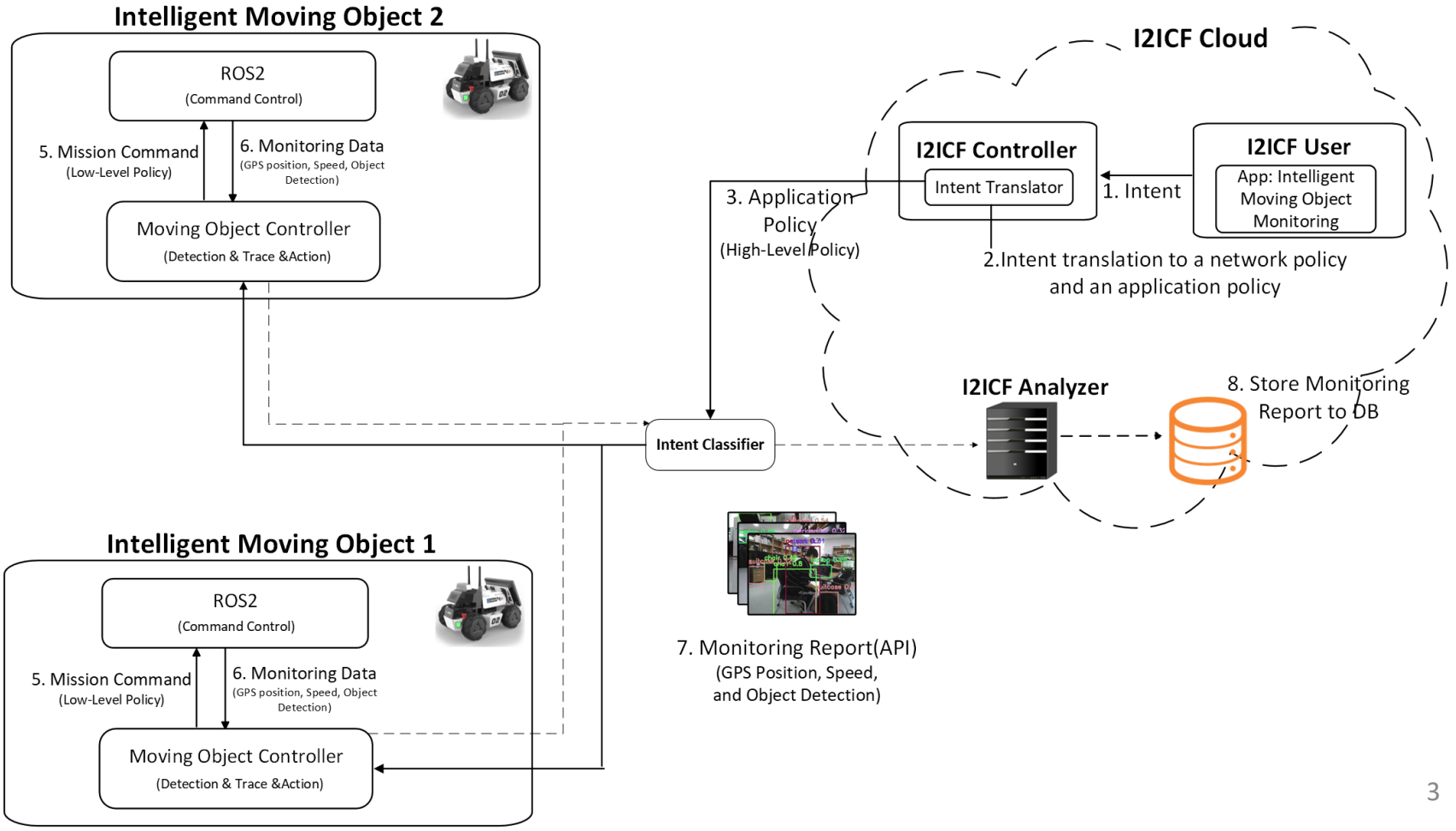
I2ICF Developing Environment

- OS: Ubuntu 20.04
- Kubernetes: Microk8s v1.32.2
- Object Detection: YOLO 11n
- LLM: Ollama (Llama 3.1 8B)
- ROS2 Version: Humble
- Backend: Django REST
- GitHub Repository: https://github.com/jaehoonpauljeong/I2ICF/tree/main/IETF-126

Workflow of the I2ICF Testbed

1. Intent Submission: A User submits a Trace/Detection/Action intent via web UI in natural language (e.g., "move forward and detour").
2. Intent Translation with LLM: The I2ICF Controller passes the intent to Ollama (Llama 3.1 8B), which outputs a structured command of {action, speed, duration} as a high-level policy.
3. Intent Classification: AgileX LIMO receives the translated intent and selects an appropriate action from the list.
4. Action: LIMO executes the selected action via rosbridge WebSocket (/cmd_vel) if a new obstacle is detected.
5. Perception & Detour: LIMO detects an object at 10 Hz using a camera with YOLO11n, selects a detour path to avoid the obstacle, and continues to move.

Interface to In-Network Computing Functions (I2ICF) for Mobile Objects

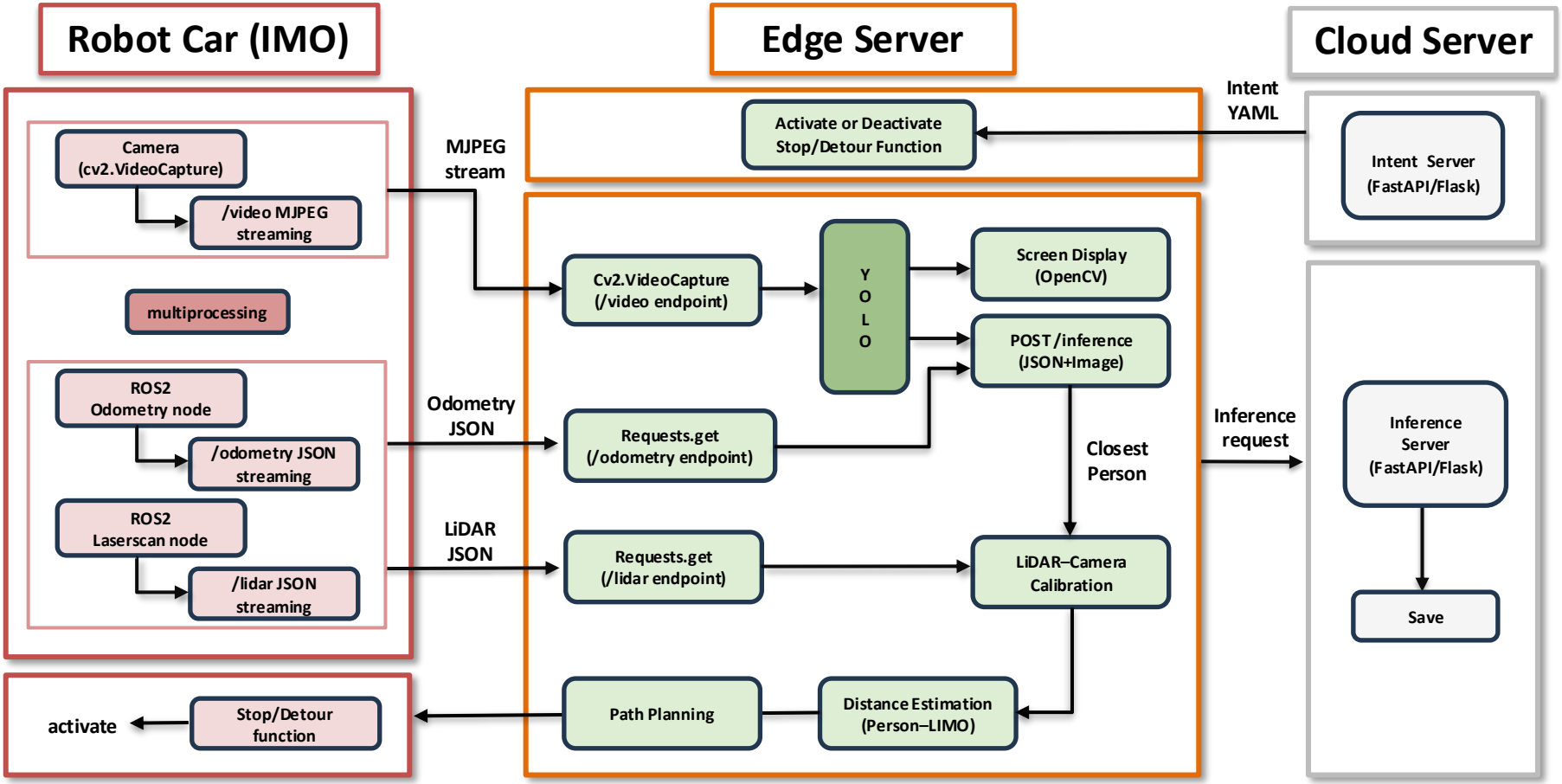


Goal of I2ICF Project

- The goal is to show the feasibility of the I2ICF framework that translates natural language intent into complex actions for a moving object
  - **I2ICF (Interface to In-Network-Computing Functions) Moving Object Controller**
    - An intent translation server converts complex user intents into a machine-readable policy document executable by the LIMO moving object and delivers it to the robot.
    - Upon receiving the translated policy document, the LIMO executes the specified actions and provides runtime status for execution monitoring
- Internet Drafts for the I2ICF Project
  - https://datatracker.ietf.org/doc/draft-jeong-nmrg-ibn-network-management-automation/07/
  - https://datatracker.ietf.org/doc/draft-ahn-nmrg-5g-security-i2nsf-framework/02/
  - https://datatracker.ietf.org/doc/draft-gu-nmrg-intent-translator/03/

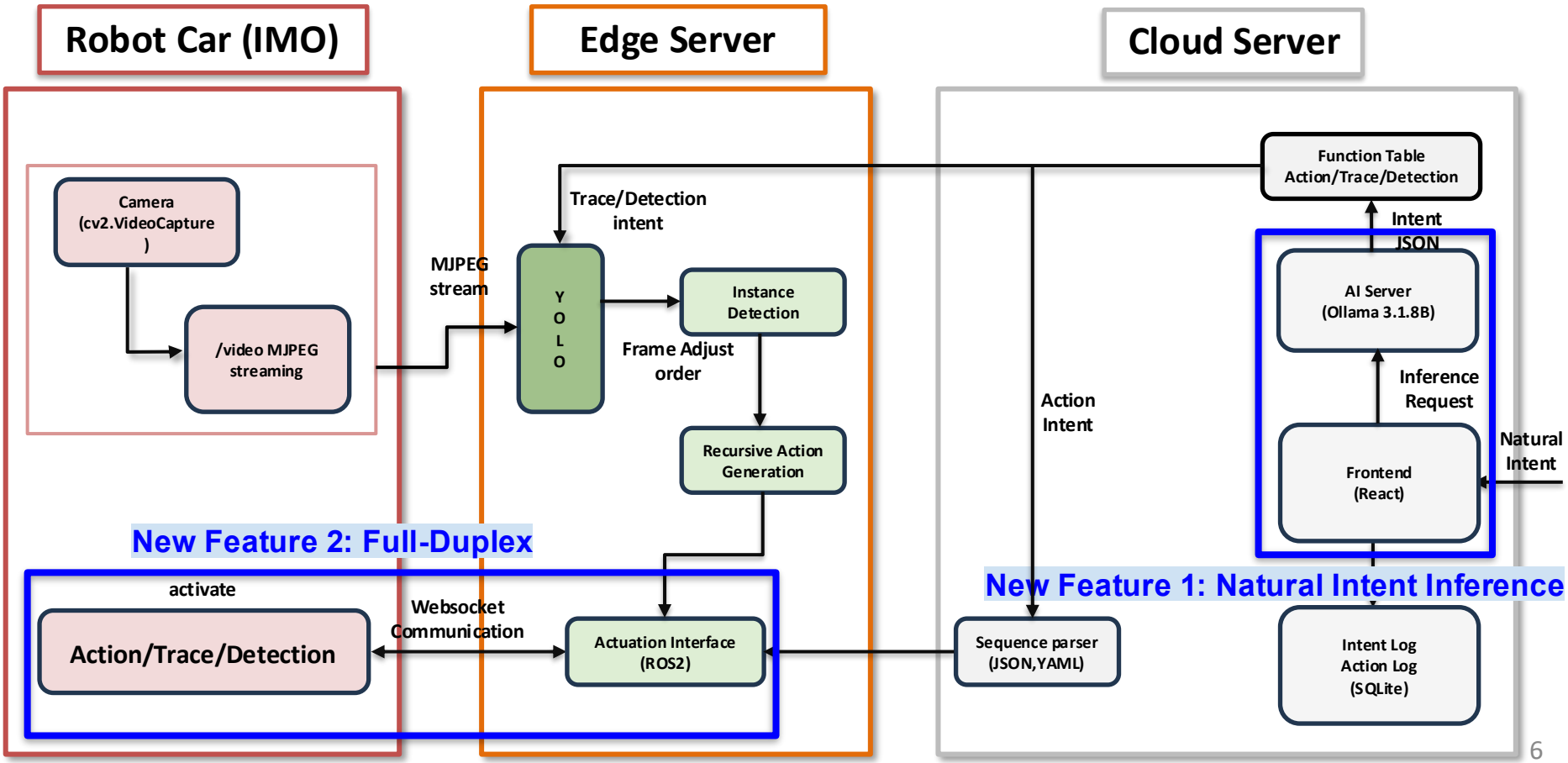
IETF-125 I2ICF Framework (Before)

: To show feasibility of self-determined collision avoidance for moving objects.

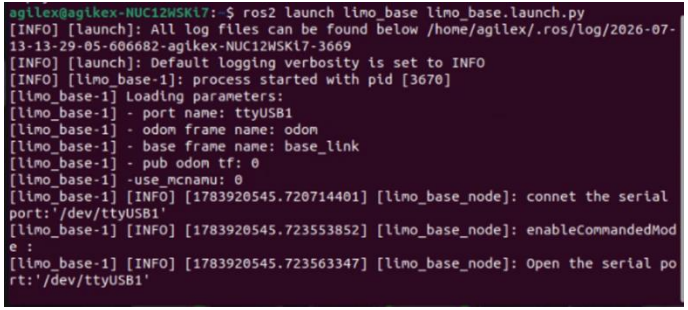


IETF-126 I2ICF Framework (Now)

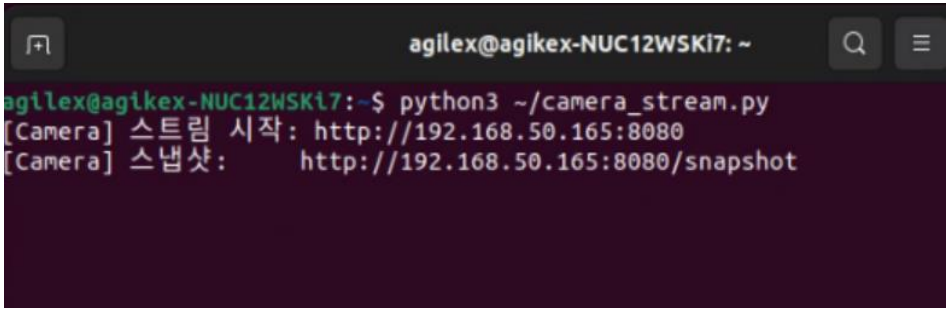
: To translate natural language intent into complex actions



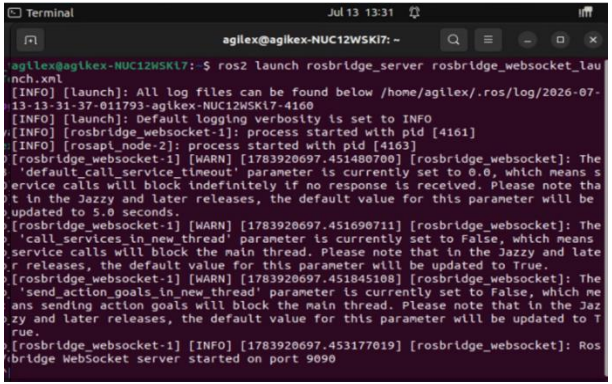
Demonstration (1/4)



1. [LIMO] Run ‘limo_base.launch.py’
(Initializes the LIMO hardware interface and enables ROS 2-based motion control)



2. [LIMO] Run ‘Camera stream.py’
(Captures camera frames and provides MJPEG streaming and snapshot endpoints)



3. [LIMO] Run ‘websocket’
(Enables real-time communication between the backend server and ROS 2 topics)

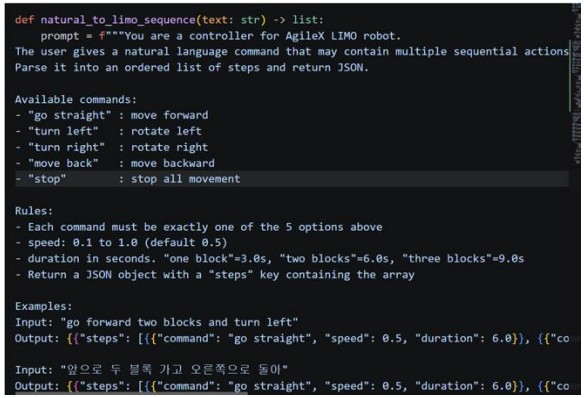
Demonstration (2/4)

User-Facing Layer

Internal Processing Layer

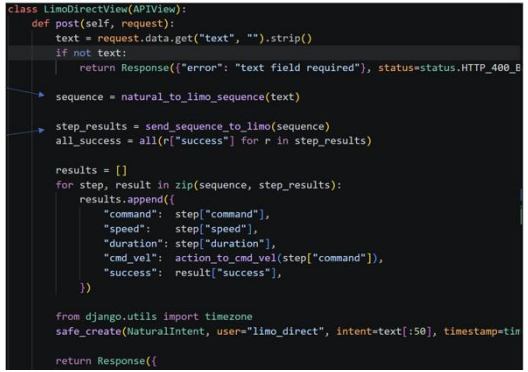
1. Natural-Language Intent Input

4. LIMO Command Execution



```
def natural_to_limo_sequence(text: str) -> list:
    prompt = f"""You are a controller for AgileX LIMO robot.
The user gives a natural language command that may contain multiple sequential actions
Parse it into an ordered list of steps and return JSON.

Available commands:
- "go straight" : move forward
- "turn left"   : rotate left
- "turn right"  : rotate right
- "move back"   : move backward
- "stop"        : stop all movement

Rules:
- Each command must be exactly one of the 5 options above
- speed: 0.1 to 1.0 (default 0.5)
- duration in seconds. "one block"=3.0s, "two blocks"=6.0s, "three blocks"=9.0s
- Return a JSON object with a "steps" key containing the array

Examples:
Input: "go forward two blocks and turn left"
Output: {{"steps": [{{"command": "go straight", "speed": 0.5, "duration": 6.0}}, {{"co

Input: "앞으로 두 블록 가고 오른쪽으로 돌아"
Output: {{"steps": [{{"command": "go straight", "speed": 0.5, "duration": 6.0}}, {{"co
```

2. LLM-Based Intent-to-JSON Conversion



```
class LimoDirectView(APIView):
    def post(self, request):
        text = request.data.get("text", "").strip()
        if not text:
            return Response({"error": "text field required"}, status=status.HTTP_400_B

        sequence = natural_to_limo_sequence(text)

        step_results = send_sequence_to_limo(sequence)
        all_success = all(r["success"] for r in step_results)

        results = []
        for step, result in zip(sequence, step_results):
            results.append({
                "command":  step["command"],
                "speed":    step["speed"],
                "duration": step["duration"],
                "cmd_vel":  action_to_cmd_vel(step["command"]),
                "success":  result["success"],
            })

        from django.utils import timezone
        safe_create(NaturalIntent, user="limo_direct", intent=text[:50], timestamp=tim

        return Response({
```

3. Robot Command Dispatch to LIMO

Demonstration (3/4)

Supported Features

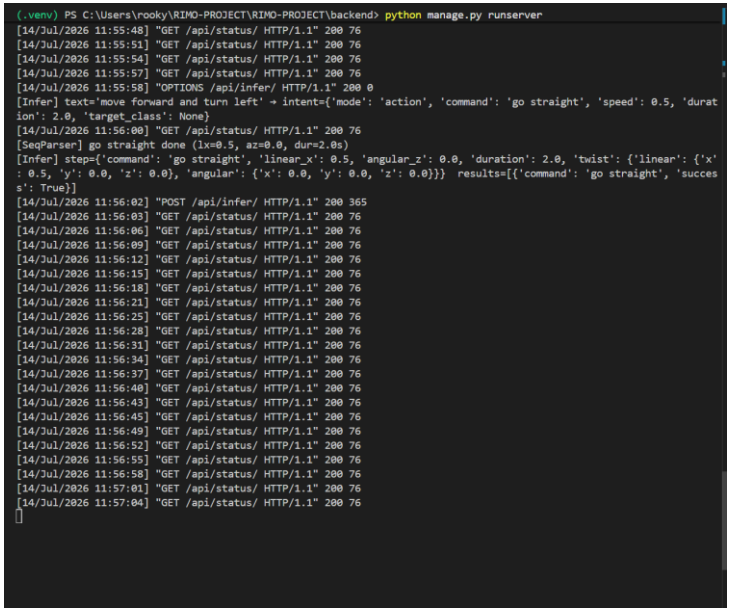
```python
class TraceSession:
    def __init__(self):
        self._running = False
        self._target_class = "person"
        self._thread: Optional[threading.Thread] = None

    @property
    def running(self) -> bool:
        return self._running

    def start(self, target_class: str = "person") -> None:
        if self._running:
            self.stop()
        self._target_class = target_class
        self._running = True
        self._thread = threading.Thread(target=self._loop, daemon=True)
        self._thread.start()
        print(f"[Trace] Session started — target: {target_class}")

    def stop(self) -> None:
        self._running = False
        if self._thread:
            self._thread.join(timeout=3)
        print("[Trace] Session stopped")
```

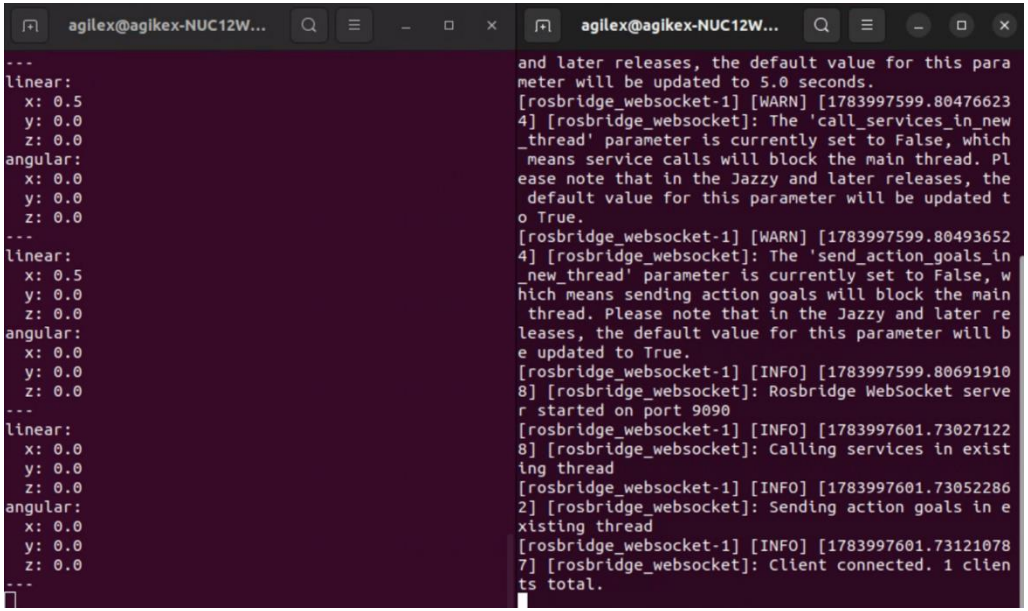
1.trace.py
(Object tracing)

```python
_SEARCH_STEP = 0.3  # seconds per search tick


class DetectionSession:
    def __init__(self):
        self._running = False
        self._thread: Optional[threading.Thread] = None

    @property
    def running(self) -> bool:
        return self._running

    def start(self) -> None:
        if self._running:
            return
        self._running = True
        self._thread = threading.Thread(target=self._loop, daemon=True)
        self._thread.start()
        print("[Detection] Session started")

    def stop(self) -> None:
        self._running = False
        if self._thread:
            self._thread.join(timeout=3)
        print("[Detection] Session stopped")

    # ── internal ──────────────────────────────────────────────────────────
```

2. detection.py
(Detect person and react)

Demonstration (4/4)





1. Backend Processing Logs
(Shows the interpreted intent, generated robot command, and execution result)

2. ROS 2 Command Reception via WebSocket
(Confirms that velocity commands are transmitted through WebSocket and received by the ROS 2 system)

What we learned

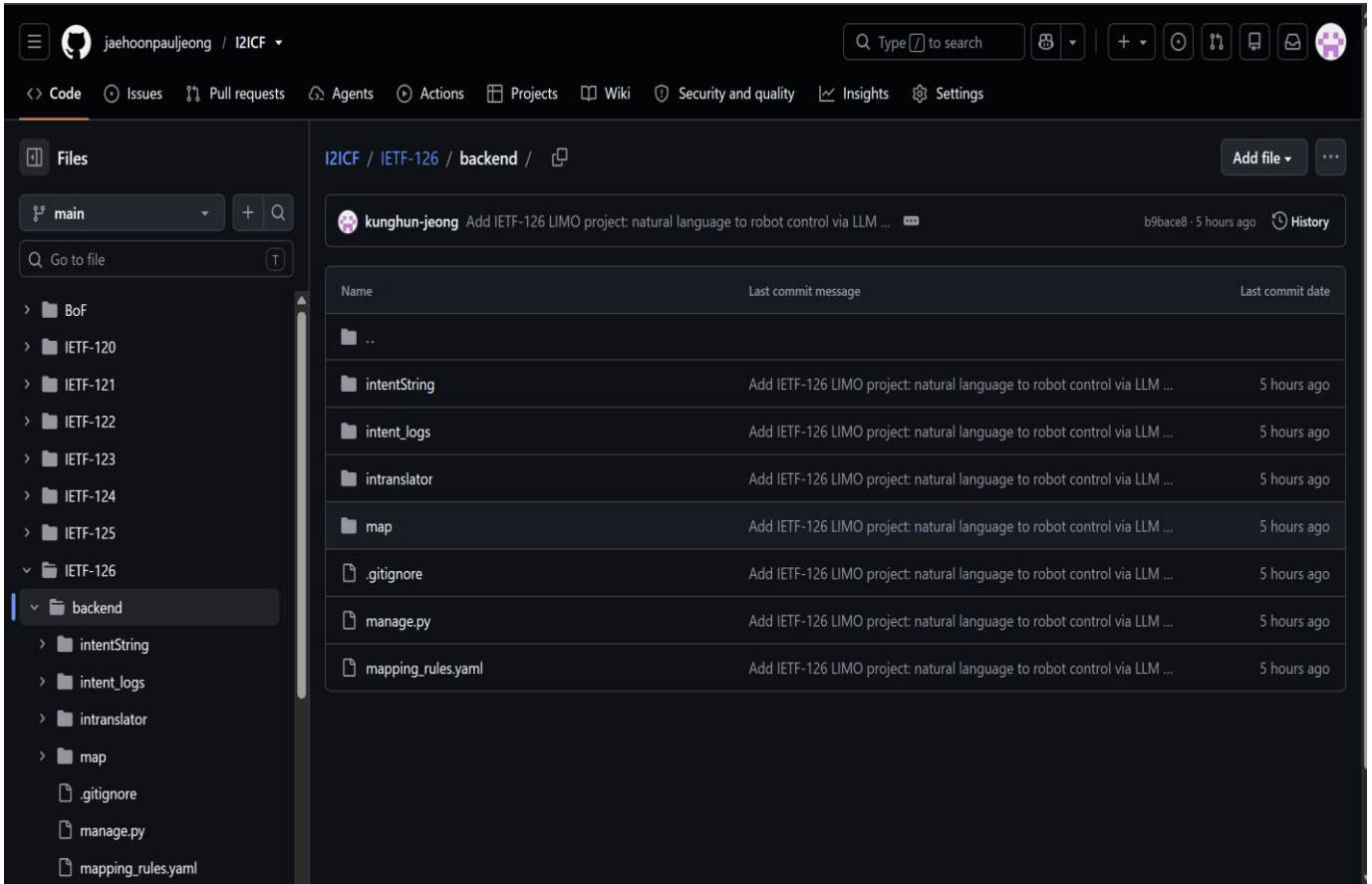
- **Enhanced LIMO's actuation based on simplified intent (e.g., Stop/Detour)**
  - We updated LIMO's intent understanding to include not only 'go', 'stop', and 'detour', but also more complex intent adaptation, such as detecting and chasing while detouring obstacles.
- **Natural Intent Translation to Policy Inference**
  - We utilized a Large Language Model (Llama 3.1) to extract key semantic representations from natural language inputs, subsequently generating a control policy to retrieve core actions from the predefined function table."
- **Integration of Full-Duplex Control Architecture**
  - We leveraged previous hackathon achivements_object detection and LiDAR distance calibration modules used for obstacle detouring and successfully integrated them into LIMO's intent-driven management system.

Next Step

- **Enhance Robustness**
  - Implement a fully automated, intent-driven management framework tailored for dynamic, moving objects.
- **Deploy an Agentic orchestrator**
  - Introduce a higher-level LLM as an agentic orchestrator to seamlessly orchestrate the entire system pipeline.
- **Optimize for Latency**
  - Execute speculative actions based on fast-path predictions while the brain AI reasons in parallel, followed by immediate command correction if any discrepancy arises.
  - Stream continuous camera frames via MJPEG from a LIMO robot car to an edge server to minimize latency for real-time YOLO perception.

Open-Source Project for IETF-126

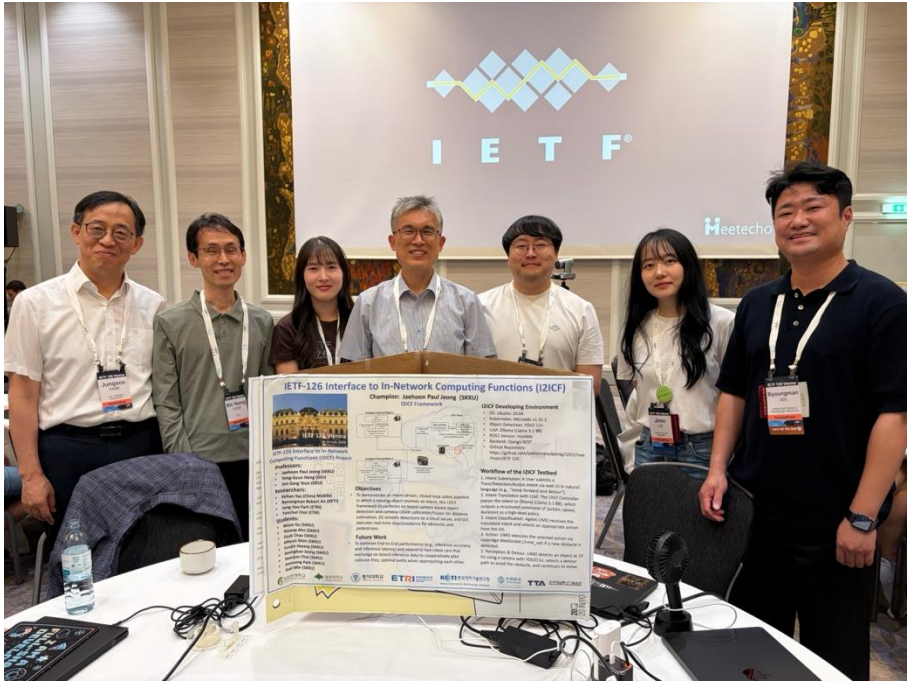
https://github.com/jaehoonpauljeong/I2ICF/tree/main/IETF-126/



I2ICF Team

- **Professors**:
  - Jaehoon Paul Jeong (SKKU)
  - Yong-Geun Hong (DJU)
  - Joo-Sang Youn (DEU)
- **Researchers**:
  - Kehan Yao (China Mobile)
  - Byoungman Robert An (KETI)
  - Jung-Soo Park (ETRI)
  - Bin Young Yoon (ETRI)
  - Yunchul Choi (ETRI)
- **Students**:
  - Mose Gu (SKKU)
  - Yoseop Ahn (SKKU)
  - Jisuk Chae (SKKU)
  - Kihyun Shim (SKKU)
  - Eunjin Hwang (SKKU)
  - Xudong Wang (SKKU)
  - Keonghun Jeong (SKKU)
  - Yoonjun Choi (SKKU)
  - Joonsang Park (SKKU)
  - Jisol Min (SKKU)

Hackathon Team Photo



Thank you!